

L4

模型微调与 私有化大模型

本讲提纲

四个模块 · 一节课讲透

PART 01

大模型开发方法论

选型 · 数据 · 评估 · 落地的系统性框架

PART 02

GPU 与国产算力生态

架构 · 市场格局 · 国产芯片 · 软件生态

PART 03

大模型多卡部署

vLLM · 并行策略 · PagedAttention · Continuous Batching

PART 04

Multi-Agent 多智能体

CrewAI · ReAct · Token 经济学

PART 01

大模型开发方法论

从选型到落地的系统性思维框架

本章内容 · 要不要用 · 技术路线选型 · 成本与复杂度 · 何时必须微调 · 数据工程 · 评估方法论 · 项目落地流程

L4 · 模型微调与私有化大模型



一、要不要用大模型

这是最容易被跳过、但最重要的第一步。

✓ 适合用大模型的场景

- 需要泛化理解
- 开放式内容生成
- 长尾情况处理
- 多步推理与决策

✗ 不适合用大模型的场景

- 任务高度结构化
- 规则明确可枚举
- 数据分布稳定
- 传统方案成本低十倍

💡 判断标准：如果不能用一页纸说清楚「输入是什么、输出是什么、怎么算好」，就别急着写代码。

📋 立项前必须回答四个问题

- ① 这个任务真的需要大模型吗？传统 NLP、规则、小模型能否搞定？
- ② 准确率目标是多少？医疗 / 金融场景 5% 错误率是灾难，客服场景或许可以接受。
- ③ 延迟和成本预算是多少？是否需要离线推理？

二、技术路线选型（由轻到重）

核心原则：够用就好。路线谱系：Prompt → RAG → Agent → Fine-tuning → 继续预训练 → 从头训练

01 Prompt Engineering

适用条件

通用知识 · 快速验证 · 试错成本低

典型场景

改写、摘要、分类、POC

定位

项目第一步，用来探天花板

02 RAG

适用条件

外部/私有知识 · 频繁更新 · 需要溯源

典型场景

知识库、客服、法律医疗、文档助手

定位

事实类任务首选，优先级高于微调

03 Agent

适用条件

多步执行 · 调用工具 · 容忍延迟

典型场景

流程自动化、订票下单、代码助手、数据分析

定位

从「回答」到「执行」的跨越

04 微调 (SFT/DPO/RLHF)

适用条件

Prompt+RAG+Agent 已不够用 · 有高质量数据 · 需固化风格 · 需降低推理成本 · 数据私有化

典型场景

特定风格输出、专业术语、成本压缩

定位

DPO/RLHF 还需偏好对比数据、提升工具调用准确率

三、成本与复杂度对比

方法	开发成本	推理成本	维护难度	效果天花板	稳定性
传统方法	中	极低	低	低（特定任务高）	高
Prompt	低	中-高	低	中-高	中
RAG	中	中-高	中	高	中-高
Agent	高	高	高	高	低-中
微调	高	低-中	高	高	中-高

颜色说明 绿色 = 有利（低成本/高稳定） 红色 = 不利（高成本/低稳定） 橙色 = 中等 黑色 = 中性

四、什么时候必须微调

有些问题，Prompt 和 RAG 本质上无法解决，微调是唯一选择。

场景 A 格式极度稳定

下游系统不容忍偶尔出错（自动扣款、病例归档、法律文档）时，Prompt「大多数时候对，偶尔出错」的特性会造成灾难。微调用 1000 条样本就能把 JSON 格式等结构化输出的准确率拉到 99.9%+

场景 B 风格 / 语气固化

法律公文的正式感、客服的品牌语气、诗歌的格律、特定作家的风格——Prompt 能模仿个大概，微调能达到「真假难辨」的水准。

场景 C 通用模型能力显著不足

小语种翻译、专业术语（医学、法律、化学）、特殊符号（LaTeX、化学式、乐谱）、视觉领域（工业缺陷、医学影像）——通用模型训练数据稀缺，再怎么 prompt 都达不到专业水平。

场景 D 需要显著降低成本

GPT-4 每天 10 万次调用约 \$1000/天，一个月 30 万；微调 Qwen-7B 达到相同效果后，一次性成本 + 推理成本可降 90%。规模化到一定程度，微调是必然选择——哪怕不是为了效果，只为了成本。

五、数据工程

01

数据质量 >> 数据数量

SFT 阶段，1 万条高质量数据通常优于 10 万条噪声数据。质量是核心变量，数量是辅助条件。

02

Few-shot 优先于微调

能用 5 个示例解决的，不要去准备 5000 条训练集。最轻量能解决的，就不要升级技术路线。

03

数据多样性审计

任务类型、难度、长度分布都要均衡。单一来源或偏斜分布会导致模型泛化能力不足。

04

评估集严格隔离

训练集和评估集不能混，最好有人工标注的 golden set。评估集污染会导致评测失真，给项目带来虚假信心。

六、评估方法论

核心原则：先定义评估，再做训练。

通用能力评估

- MMLU (综合知识)
- GSM8K (数学推理)
- HumanEval (代码生成)
- BBH (复杂推理)

对齐质量评估

- MT-Bench (多轮对话)
- AlpacaEval (指令跟随)
- Arena (模型竞技场)

业务指标评估

- 自定义领域 Benchmark
- 决定能否上线的核心标准
- 结合业务场景构建专属评测集

安全性评估

- 拒答率检测
- 越狱测试
- 偏见与公平性测试

 线上配合：A/B 测试 + 用户反馈收集 (thumbs up/down) → 形成数据飞轮，持续优化。

七、项目落地流程（工程视角）



💡 此流程非线性——评估失败时随时回退至上一阶段；数据飞轮是持续改进而非一次性工程。

PART 02

GPU 与国产算力生态

从英伟达到昇腾 · 看懂中国算力的真实地图

本章内容 · GPU 架构 · 市场格局 · 国产芯片 · 软件生态 · 迁移实践

L4 · 模型微调与私有化大模型



英伟达主流 GPU 对比

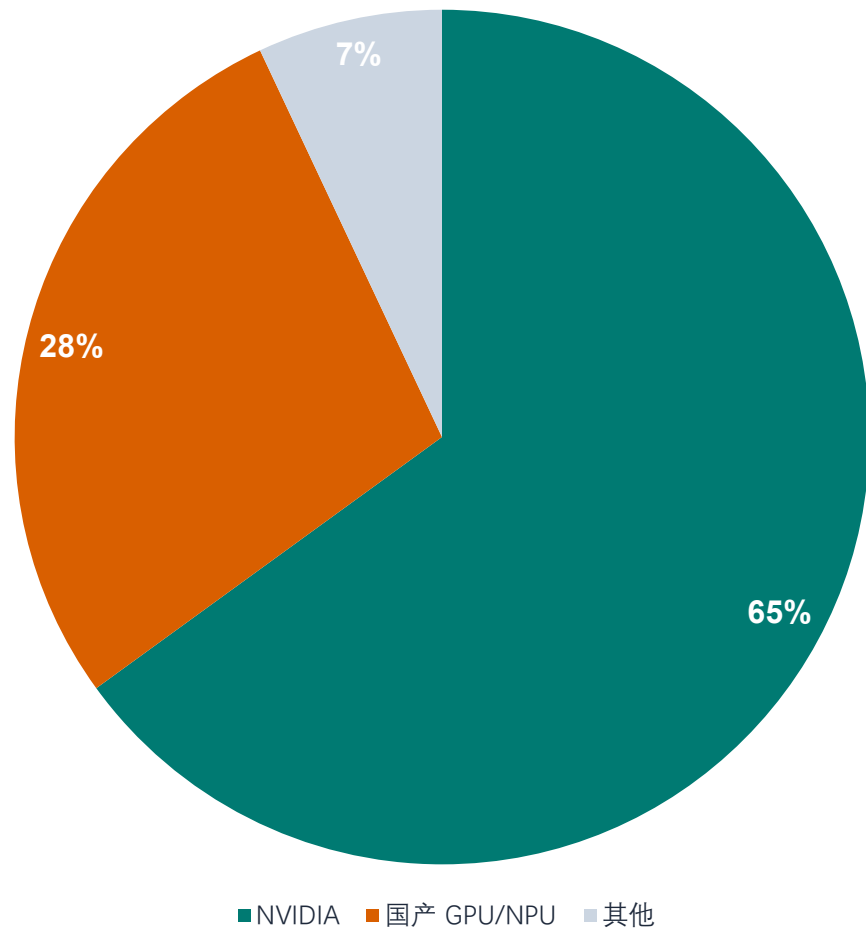
NVIDIA GPU Lineup at a Glance

GPU 型号	GPU 类型	FP16/BF16 算力	显存	市场保有量	定位
H100 / H200	数据中心 · 训练	~1000 TFLOPS	80 GB	★	最强算力，数量最少
A100 / A800	数据中心 · 训练/推理	~300 TFLOPS	40/80 GB	★★★★	真正"干活最多"的 GPU
L40S	数据中心 · 推理/多模态	~350 TFLOPS	48 GB	★★★	企业部署主力
RTX 4090/3090	消费级 · 教学/科研	~330 TFLOPS	24 GB	★★★★★	市场最多，性价比最高
Jetson Orin	边缘 · 嵌入式	~10–20 TFLOPS	8–64 GB	★★★★	藏在设备里的 AI

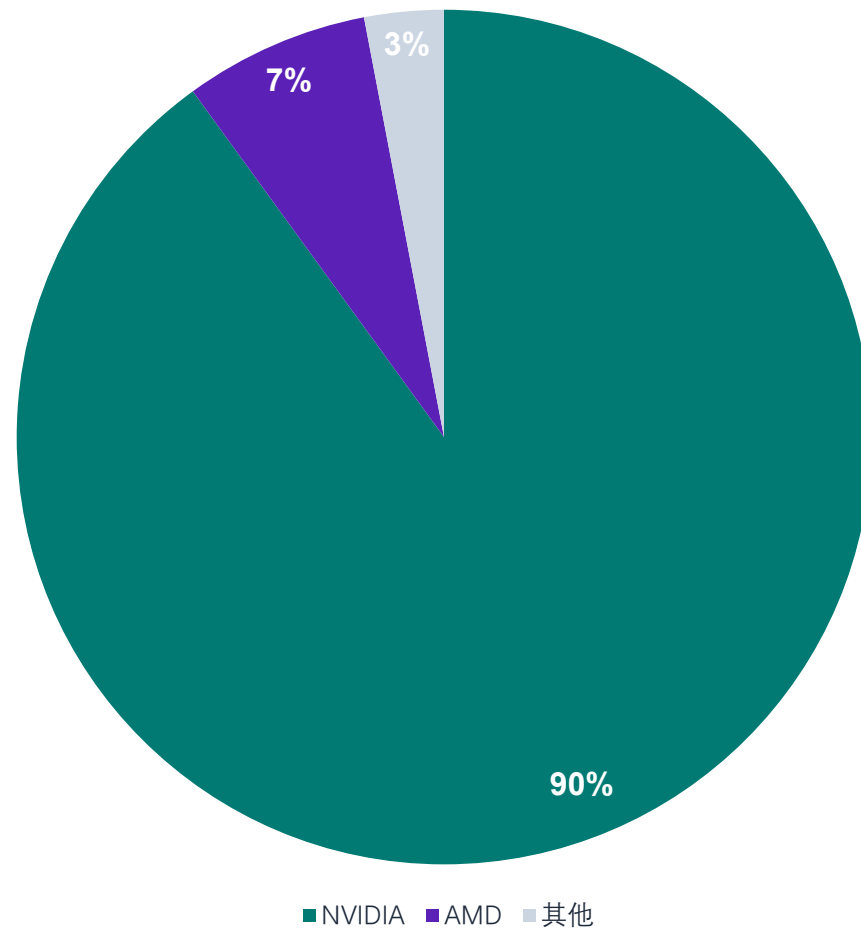
GPU 市场格局

中国 vs 全球市场份额对比

中国市场



全球市场



中国市场 AI 芯片份额详解

国产芯片已接近三分之一

阵营	市场份额	说明
NVIDIA（含合规型号）	60 – 70%	仍是最大单一厂商
国产 AI 芯片（合计）	25 – 35%	快速上升中
其他（AMD / Intel 等）	< 5%	可忽略不计

国产 AI 芯片已接近「三分之一」的中国市场

国产阵营内部分布

厂商	国产内部占比	整体市场
华为 昇腾（Ascend）	60 – 70%	15 – 25%
寒武纪（Cambricon）	10 – 15%	3 – 5%
百度 昆仑芯	5 – 10%	2 – 3%
海光 DCU	5 – 10%	2 – 3%
壁仞 / 天数 / 其他	5 – 10%	< 3%

GPU vs 国产 AI 芯片

定义与核心差异

核心结论：国产主流 AI 芯片并非 GPU，而主要属于 NPU / AI Accelerator（ASIC），专为神经网络计算而设计。

NVIDIA GPU

GPU 是什么？

- 通用并行计算处理器
- 最初用于图形渲染
- 广泛用于 AI 矩阵计算
- 通用性强，图形+AI+HPC
- 代表：NVIDIA

NPU / ASIC

国产主流 AI 芯片

- 走 NPU / ASIC 路线
- 只做 AI，不做图形
- 特定神经网络效率高
- 功耗低、可控性强
- 华为昇腾、寒武纪 MLU、百度昆仑

国产 GPU

国产 GPU（少量）

- 定位通用 GPU 或 AI GPU
- 技术难度高，仍在追赶
- Moore Threads 摩尔线程
- Jingjia Micro 景嘉微
- Biren Technology 壁仞

华为 vs 英伟达：昇腾 910C 对比 H100

算力 · 价格 · 软件生态

华为 昇腾 910C

算力

H100 的 60~80%

价格

~18-20 万 RMB / 卡

编程框架

CANN + MindSpore
PyTorch 适配成本高

大模型训练

需工程调优，迁移成本较高

英伟达 H100

基准 (FP16 ~2,000 TFLOPS)

~17-21 万 RMB / 卡

CUDA 事实标准
PyTorch 原生支持

开箱即用，社区生态最成熟

VS

国产 AI 芯片软件生态

NVIDIA CUDA 生态 vs 国产专用生态

国外 GPU (NVIDIA)

- CUDA: 训练/推理/HPC 的事实标准
- 工具链完善: cuDNN · TensorRT · NCCL · Profiler
- 框架支持: PyTorch / TensorFlow 即装即用
- 代码迁移成本低, 社区文档极其成熟
- [优势] 通用性强 · 学习成本低 · 生态最成熟

国产 AI 芯片 (NPU/ASIC)

- 自有编程模型/编译器 (非 CUDA)
- 需专用插件或适配层才能运行
- 算子与模型适配成本高
- 工具链在完善中, 但统一性不足
- [特点] 效率高·可控, 但通用性与成熟度较弱
- 注: 海光 DCU · 芯动 GPU 兼容类 CUDA 生态

英伟达 GPU 大模型迁移到华为昇腾

哪些要改？ 哪些不用改？

不需要改

- 模型权重 / 结构
- Tokenizer / 词表
- LoRA / Adapter 权重
- 训练数据与数据格式

必须改

- CUDA/cuDNN → CANN
- PyTorch → torch-npu
- 设备: cuda → npu
- torch.cuda.* → NPU 接口
- CUDA AMP → FP16/BF16 (Ascend)

重点工程改动

- FlashAttention/xFormers/Triton → 不可直接用
- bitsandbytes (4/8bit量化) → 不兼容
- 处理: 回退标准 PyTorch 或用 Ascend 实现
- — 推理部署 —
- TensorRT/CUDA vLLM → 不能用
- 改用: vLLM-Ascend / PyTorch-NPU

主流国产 GPU / NPU 量化支持

精度支持 · 量化方案 · 推理框架

厂商 / 芯片	原生支持精度	量化方案	推理框架
寒武纪 MLU	FP16/BF16/INT8/FP8	MagicMind；FP8+INT4 混合量化	vLLM-MLU (fork) · MagicMind
华为 昇腾	FP16/BF16/INT8/FP8	昇腾自有量化工具链	vLLM-Ascend · SGLang 适配版
海光 DCU	FP16/BF16/INT8	接近 AMD ROCm 生态	原生 vLLM 基本兼容
摩尔线程	FP16/FP8	基于 vLLM；新代 GPU 原生 FP8	vLLM 适配版
壁仞 BW3000/Z100	FP16	lmslim 库；AWQ 仅 K100AI 支持	vLLM（部分支持）

国产 AI 芯片主要工具链

与 CUDA 生态的对应关系

厂商 / 芯片	芯片类型	核心工具链	对标对象	PyTorch	生态成熟度
华为 昇腾 Ascend	NPU	CANN	CUDA + cuDNN	适配版	★★★★★
海光 DCU	GPU/DCU	DTK	ROCm	ROCm	★★★★★
沐曦 MTT	GPU/DCU	DTK / MX-Toolchain	ROCm	支持	★★★★
寒武纪 MLU	AI ASIC	Neuware	CUDA	适配	★★★★
百度 昆仑芯	AI ASIC	Kunlun SDK / XPU	CUDA	适配	★★★★
壁仞 BR100	GPU	BR-Toolchain	CUDA	部分	★★★
燧原科技	AI ASIC	Tops SDK	CUDA	部分	★★★
天数智芯	GPU	DTU SDK	CUDA	部分	★★★

PART 03

大模型多卡部署

— 以 vLLM 为例

本章内容 · 四种并行 · PagedAttention · Continuous Batching · vLLM 多卡部署执行栈 · 策略选择

为什么大模型需要多卡部署？



模型权重显存

FP16/BF16 下，参数显存 $\approx$ 参数量 $\times$ 2 bytes

70B 模型仅权重就 $\approx$ 140 GB，远超单张 80 GB GPU。

→ 单卡装不下



KV Cache 显存

自回归生成时每个 token 都要缓存 Key/Value，并发越高、上下文越长，占用越大。

→ 显存碎片 + 重复存储 → 严重浪费

多卡部署要解决五件事

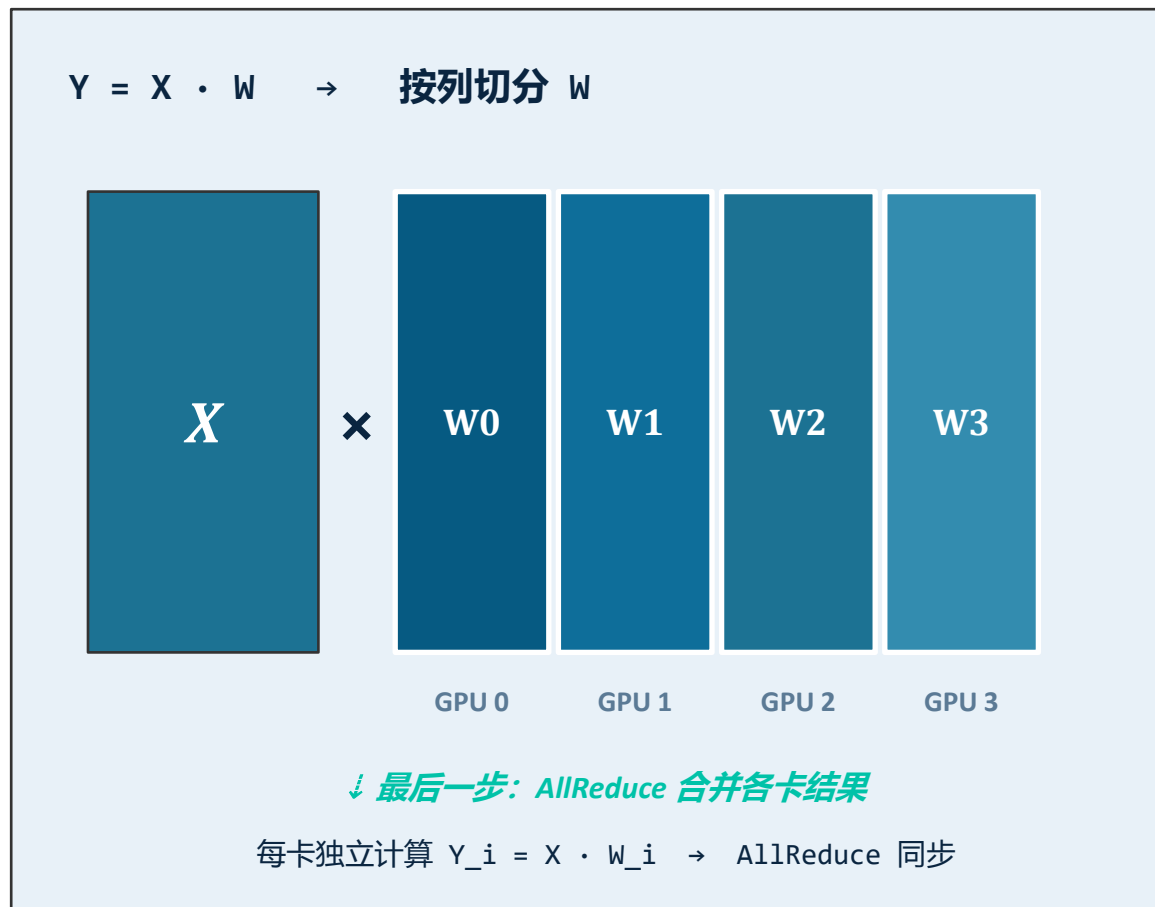
- ✓ **装得下** 支持更大模型权重
- ✓ **看得远** 支持更长上下文
- ✓ **接得多** 支持更多并发请求
- ✓ **跑得快** 提升整体吞吐量
- ✓ **等得少** 降低平均等待时延

四种并行策略 · 一张图看懂

 TP	 PP	 DP	 EP
Tensor Parallelism 张量并行 解决 单层矩阵太大 把每一层内部的大矩阵按行/列切分到多张 GPU，每卡只算自己那一片。	Pipeline Parallelism 流水线并行 解决 模型层数太多 按层切分，不同 GPU 负责不同层，请求像流水线一样依次穿过。	Data Parallelism 数据并行 解决 并发请求太多 每组 GPU 都部署完整模型副本，不同请求分发到不同副本独立处理。	Expert Parallelism 专家并行 解决 MoE 专家太多 把不同 expert 放在不同卡，通过路由把 token 送到对应专家。
适用场景 单机多卡 · NVLink	适用场景 跨节点 · 超大模型	适用场景 高 QPS 在线服务	适用场景 Mixtral · DeepSeek-MoE

实际部署中四种方式常组合使用：典型组合是 节点内 TP + 节点间 PP / DP；MoE 模型再叠加 EP。

张量并行 —— 把一层矩阵横向拆开



✓ 优点

- 每卡只持有部分权重，单卡显存压力小
- 每层可并行计算，延迟低
- 单节点多卡部署的首选

✗ 缺点

- 每层都需要一次 AllReduce 通信
- 强依赖 NVLink / NVSwitch，跨节点性能急剧下降
- TP size 越大，通信成本越高

启动命令

```
vllm serve facebook/opt-13b \
  --tensor-parallel-size 4
```

流水线并行 —— 把模型按层纵向切开

Input tokens



Output logits

✓ 优点

- 可跨节点扩展
- 每卡只保存部分层，省显存
- 通信量小（只在切分点传激活）
- 适合超大模型部署

✗ 缺点

- 存在 pipeline bubble（气泡）
- 单请求延迟可能增加
- 小 batch 或低并发下利用率低
- 需要 micro-batch 填满流水线

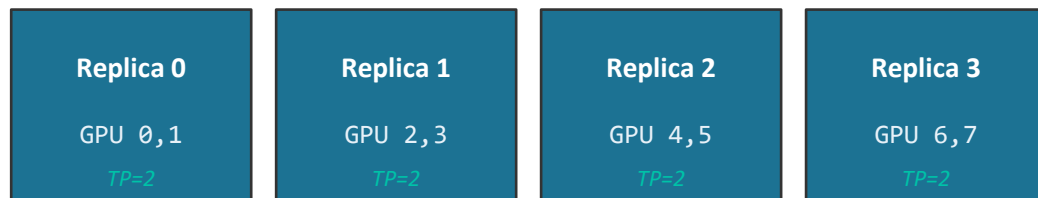
启动命令 · TP+PP 组合

```
vllm serve gpt2 \  
  --tensor-parallel-size 4 \  
  --pipeline-parallel-size 2
```

Total GPUs = TP × PP = 8

数据并行与专家并行 —— 横向扩吞吐 / MoE 专属

Data Parallelism · 数据并行 (DP)

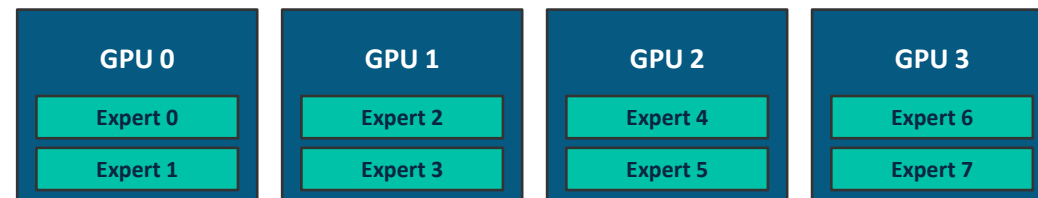


→ 每副本独立处理不同请求, 互不干扰

适用场景

- 单副本能放下, 但请求量大 / QPS 高
- 在线高并发服务横向扩容
- 可与 TP 组合: `--data-parallel-size 4 --tensor-parallel-size 2`

Expert Parallelism · 专家并行 (EP)



→ Token 按路由分发到对应专家所在 GPU

适用场景

- Mixtral / DeepSeek-MoE / Qwen-MoE 等专家模型
- 常见组合: attention 走 DP, expert 走 EP
- 启用方式: `--enable-expert-parallel`



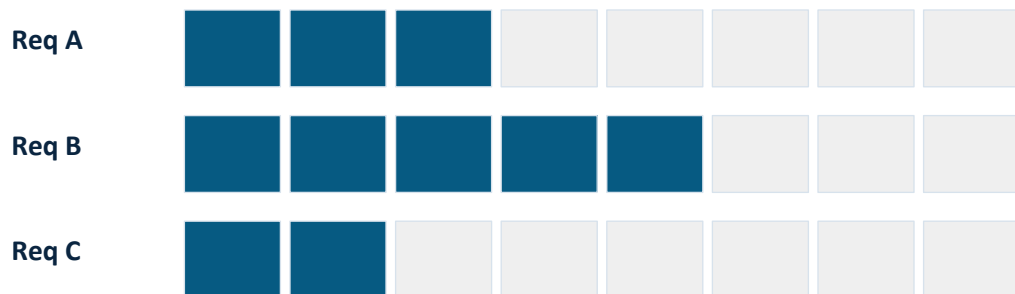
PagedAttention —— 把 KV Cache 像虚拟内存一样分页

60-80%

传统连续分配的 KV Cache 显存浪费率 → PagedAttention 几乎为零

传统分配方式

按最大可能长度预留连续显存



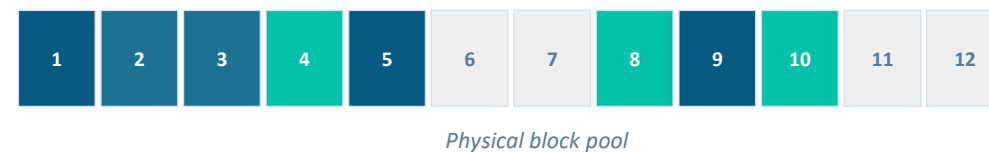
阴影部分为已浪费显存 (内部 + 外部碎片)

内部碎片: 实际长度 << 预留长度

外部碎片: 长度不一导致大量空隙无法利用

PagedAttention 分页方式

KV 切成固定 block, 通过 block table 映射



Req A → block 1 → 5 → 9

Req B → block 2 → 3

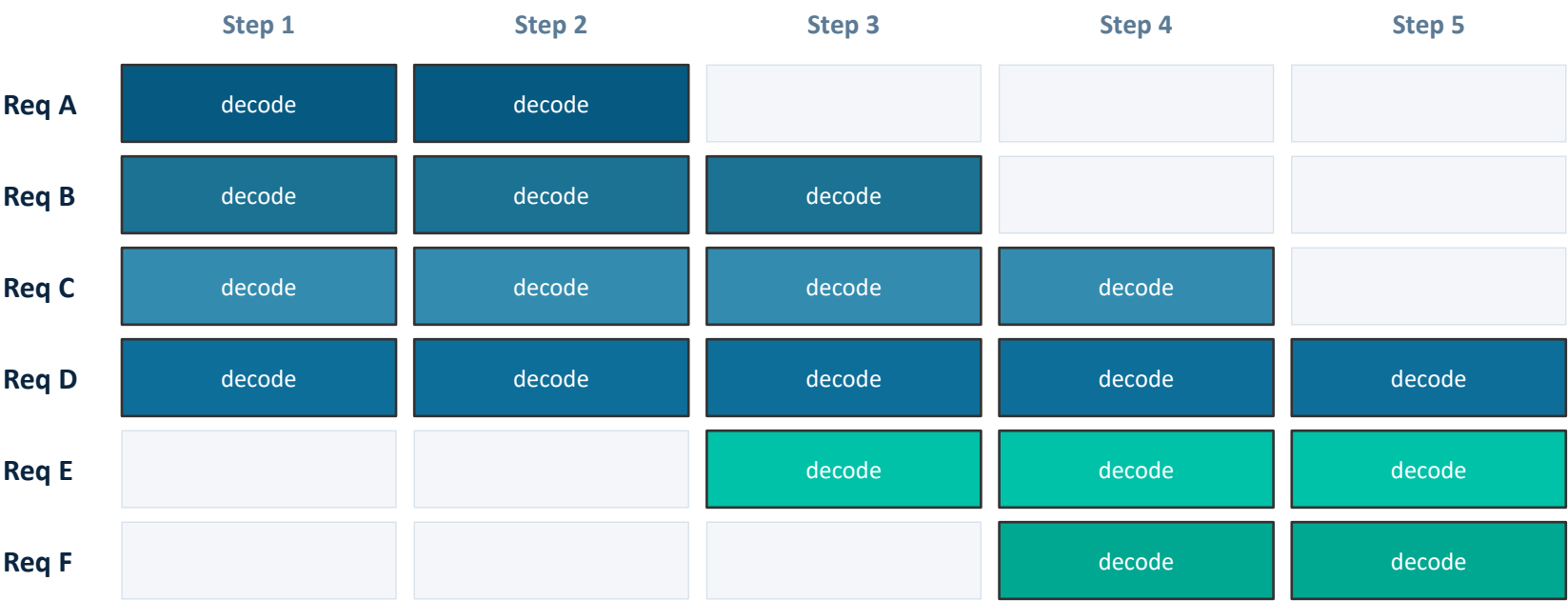
Req C → block 4 → 8 → 10

✓ 几乎零碎片 ✓ 支持 prefix sharing ✓ 大幅提高并发

Continuous Batching —— 让 GPU 永远满载

普通 batching：等一整批跑完才能开始下一批 —— 短请求陪长请求空跑。

Continuous batching：每步 decode 后重新调度，完成即出，新到即入。

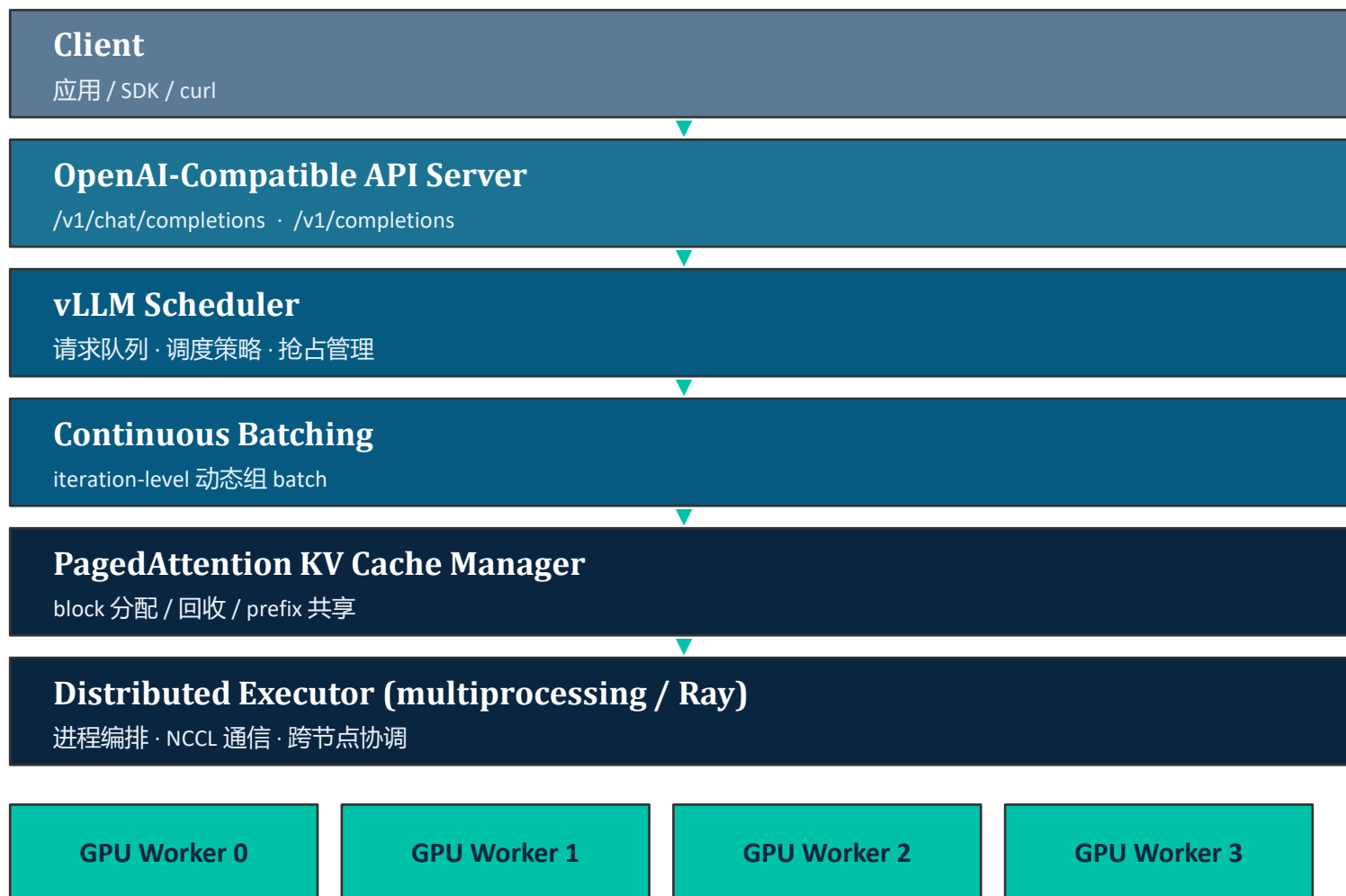


收益

- GPU 持续满载
- 尾延迟显著降低
- 适合动态请求流

Step 3: A 完成, 立即加入 E · Step 4: B 完成, 立即加入 F

vLLM 多卡部署执行栈



关键执行后端

单节点

Python multiprocessing
每 GPU 一个 worker 子进程

多节点

Ray 集群调度
--distributed-executor-backend ray

通信

NCCL · NVLink / NVSwitch / IB
TP 强依赖卡间高带宽

数据并行模式

每 DP rank 独立 engine
ZMQ 与前端通信 · 独立 KV Cache

三种典型部署场景 · 命令直接抄

场景 1

单机 4 卡，模型单卡放不下

Qwen2.5-32B · 4×A100 80GB

并行策略

$TP = 4$

\$ 启动命令

```
CUDA_VISIBLE_DEVICES=0,1,2,3 \  
vllm serve /models/Qwen2.5-32B-Instruct \  
  --host 0.0.0.0 --port 8000 \  
  --tensor-parallel-size 4 \  
  --max-model-len 32768 \  
  --gpu-memory-utilization 0.90
```

场景 2

单机 8 卡，TP + PP 混合

通信受限或更大模型 · 8 GPUs

并行策略

$TP = 4 \cdot PP = 2$

\$ 启动命令

```
vllm serve /models/large-model \  
  --tensor-parallel-size 4 \  
  --pipeline-parallel-size 2 \  
  --max-model-len 32768 \  
  --gpu-memory-utilization 0.90
```

场景 3

两节点共 16 卡，超大模型

DeepSeek-V3 / 405B 级别

并行策略

$TP = 8 \cdot PP = 2 \cdot Ray$

\$ 启动命令

```
vllm serve /models/huge-model \  
  --tensor-parallel-size 8 \  
  --pipeline-parallel-size 2 \  
  --distributed-executor-backend ray
```


我该用哪种并行？一张表决定

你的情况	推荐策略	示例配置	关键参数
单卡能放下，低并发	单卡	TP = 1	—
单卡放不下，单机能放下	TP	TP = 4 / TP = 8	--tensor-parallel-size 4
单机也放不下，需多节点	TP + PP	TP = 8, PP = 2	+ --pipeline-parallel-size 2
模型能放下，请求量大	DP	DP = 4	--data-parallel-size 4
高并发，单副本需多卡	DP + TP	DP = 4, TP = 2	共需 8 张 GPU
MoE 模型 (Mixtral / DeepSeek)	DP + EP / TP	attention DP + expert EP	--enable-expert-parallel

关键性能参数

--gpu-memory-utilization 建议 0.85–0.95 --max-model-len 上下文越长 KV Cache 越大、并发越低 --served-model-name 对外暴露的模型名

多卡部署的本质

把“模型权重、计算、KV Cache 和请求流量”——在GPU 与机器之间合理拆分。

TP

解决单层矩阵太大

PP

解决模型层数太多

DP

解决高并发请求

EP

解决 MoE 专家分布

PagedAttention

解决 KV Cache 浪费

Continuous Batching

解决 GPU 利用率不足

Ray / multiprocessing

多进程 / 多节点调度

实战速查 · QUICK RECIPE

单机多卡 → TP · 多机多卡 → TP + PP · 高并发服务 → DP + TP · MoE 模型 → DP + EP / TP



PART 04

Multi-Agent 系统

— 从概念到工程实践 · 以 CrewAI 为例

本章内容 · Multi-Agent 基础 · CrewAI 实战 · 运作机制 · Token 经济学

L4 · 模型微调与私有化大模型



课程大纲

四个模块,层层递进



智泊AI
ZHIPU AI



魔泊云
MoPaaS

01



Multi-Agent 基础

概念、架构模式、主流框架对比

02



CrewAI 实战

核心概念、完整代码示例

03



运作机制

ReAct 循环、LLM 调用、协作时序

04



Token 经济学

Prompt 拼装、成本分析、优化策略

PART 04.1

Multi-Agent 基础

概念 · 架构 · 框架对比



什么是 Multi-Agent?

从单 Agent 到多 Agent 的演进

定义

由多个能够自主决策、相互协作或竞争的智能体共同组成的系统。
每个 Agent 都有自己的目标、能力和决策逻辑。

LLM 时代的 Agent 构成

- 大模型
- 工具调用能力
- 记忆
- 角色设定



一个简单的多 Agent 团队

1 规划者
拆解任务

2 执行者
调用工具

3 审查者
评估结果

四种典型架构模式

选择合适的协作方式



智泊AI
ZHIPO AI



魔泊云
MoPaaS



层级式

Hierarchical

主控 Agent 分派任务给下属。
类似项目经理带团队。



对等式

Peer-to-Peer

Agent 地位平等,通过消息传递自主沟通。



流水线

Pipeline

按固定顺序传递结果,前者输出是后者输入。



辩论式

Debate

多 Agent 独立回答,再讨论或投票收敛结果。

💡 **实践提示:**层级式是 Anthropic Research 系统的选择;CrewAI 默认 sequential(类流水线),也支持 hierarchical。

为什么需要 Multi-Agent?

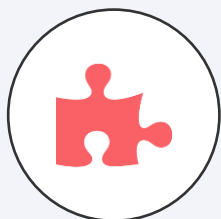
单 Agent 力不从心的四个场景



智泊AI
ZHIPU AI

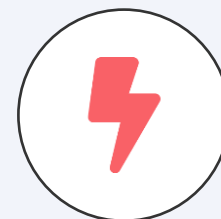


魔泊云
MoPaaS



关注点分离

每个 Agent 只需精通自己的角色,提示词更聚焦



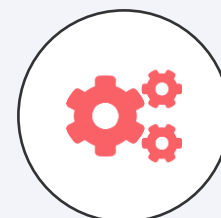
并行加速

独立子任务可同时推进,显著缩短端到端耗时



上下文隔离

避免单个上下文窗口被无关信息污染



专业化分工

不同 Agent 可用不同模型、不同工具组合

经验法则: 只有当任务确实需要并行或多视角时才上 Multi-Agent。

主流框架对比

三个梯队,各有所长



框架	梯队	特点	最适合
LangGraph	🏆 一梯队	图状态机编排,生态完善	长期生产项目
CrewAI	🏆 二梯队	角色驱动,API 简洁	快速原型验证
AutoGen	🏆 二梯队	微软出品,对话驱动	Azure 生态
MetaGPT	🏆 三梯队	模拟软件公司分工	代码生成场景
Claude Agent SDK	🏆 三梯队	Anthropic 自家,深度调优	Claude 重度用户
OpenAI Agents SDK	🏆 三梯队	官方背书,轻量	OpenAI 生态用户

本课程聚焦 CrewAI —— 因为它的概念最清晰,最适理解多 Agent 系统的本质机制。

PART 04.2

CrewAI 实战

核心概念 · 完整代码



CrewAI 四大核心概念



理解了这四个,就理解了 CrewAI

01

Agent 智能体

由 role(角色)、goal(目标)、backstory(背景)三件套定义。背景故事不是装饰 —— 它会被注入到 prompt 里,实质性影响 Agent 行为。

02

Task 任务

由 description、expected_output、agent 定义。context=[other_task] 是协作的关键 —— 自动传递上游产出。

03

Crew 团队

组装 Agent 和 Task。Process.sequential 顺序执行;Process.hierarchical 由 Manager Agent 自动调度。

04

kickoff() 启动

传入 inputs 字典,变量会替换 description 里的 {topic} 等占位符,然后整个 Crew 开始运行。

示例:研究 + 写作团队

两个 Agent 协作完成技术简报



场景设定

针对某个技术话题,先调研、再撰写一篇简报。两个 Agent 分工协作:

研究员

搜集资料,提炼信息

✓ 联网搜索工具

撰稿人

基于资料撰写简报

× 无工具

核心代码:定义 Agent

```
from crewai import Agent, Task, Crew

researcher = Agent(
    role="资深技术研究员",
    goal="搜集权威资料",
    backstory="十年经验...",
    tools=[search_tool],
    verbose=True
)

writer = Agent(
    role="技术撰稿人", ...
)
```

定义 Task 与组装 Crew

context 是协作的关键



第 1 步:定义两个 Task

```
research_task = Task(
    description="针对话题 [{topic}] 进行深入调研...",
    expected_output="结构化笔记 + 5 个信息源",
    agent=researcher,
)

writing_task = Task(
    description="基于调研笔记撰写简报...",
    agent=writer,
    context=[research_task], # 关键:产出自动传递
```

第 2 步:组装 Crew 并启动

```
crew = Crew(
    agents=[researcher, writer],
    tasks=[research_task, writing_task],
    process=Process.sequential,
)
result = crew.kickoff(inputs={"topic": "Multi-Agent"})
```

PART 04.3

运作机制

ReAct 循环 · LLM 调用 · 协作时序

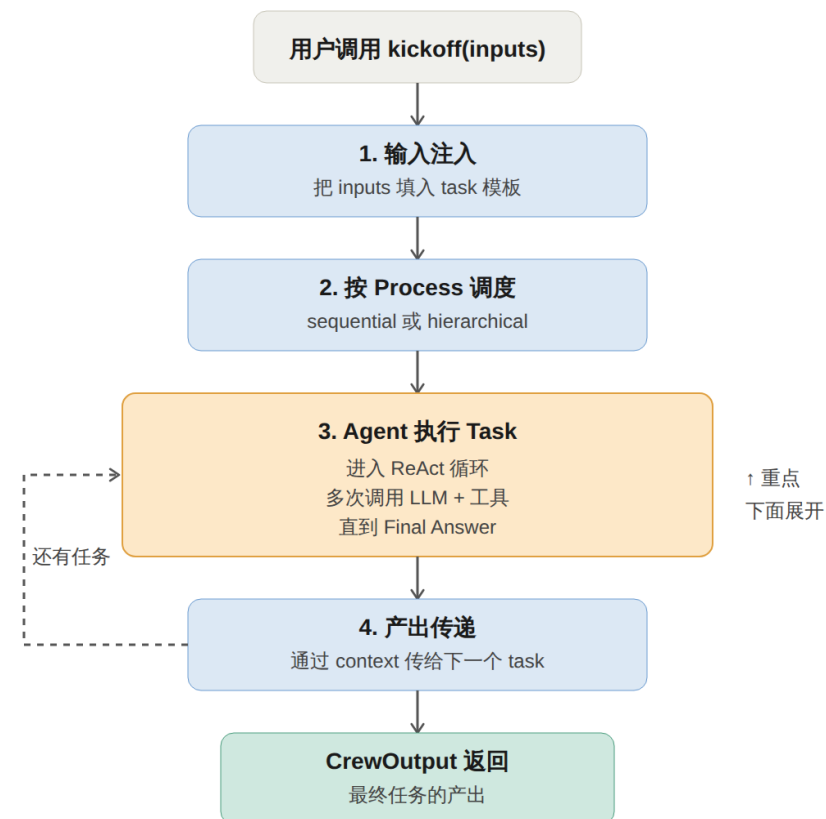


整体执行流程

kickoff 之后,内部发生了什么



- 1 输入注入**
把 inputs 填入 task 模板
- 2 按 Process 调度**
sequential 或 hierarchical
- 3 Agent 执行 Task**
进入 ReAct 循环(多次调用 LLM)
- 4 产出传递**
通过 context 传给下一个 task



ReAct 循环 —— Agent 的“心跳”

Reasoning + Acting + Observation

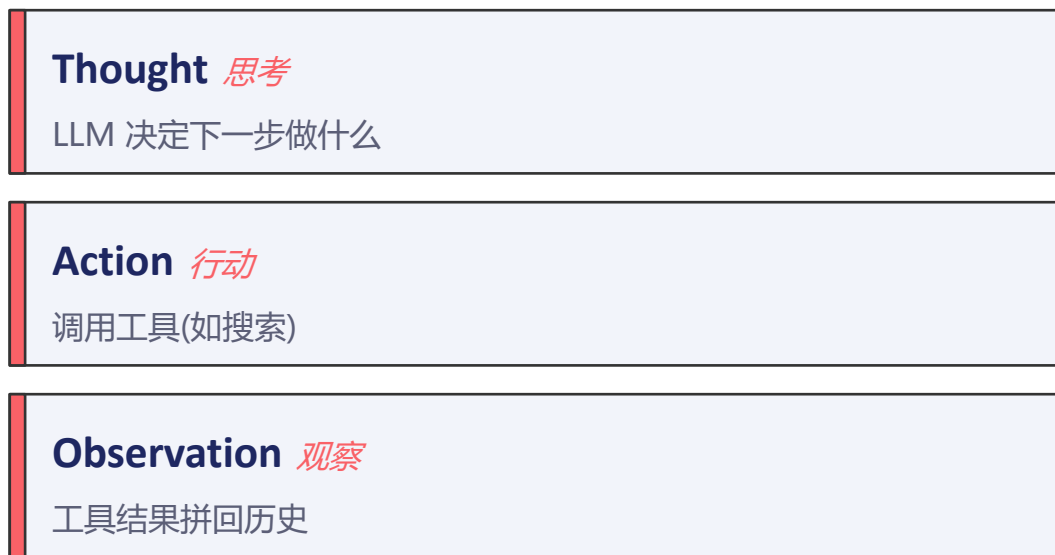


智泊AI
ZHIPO AI

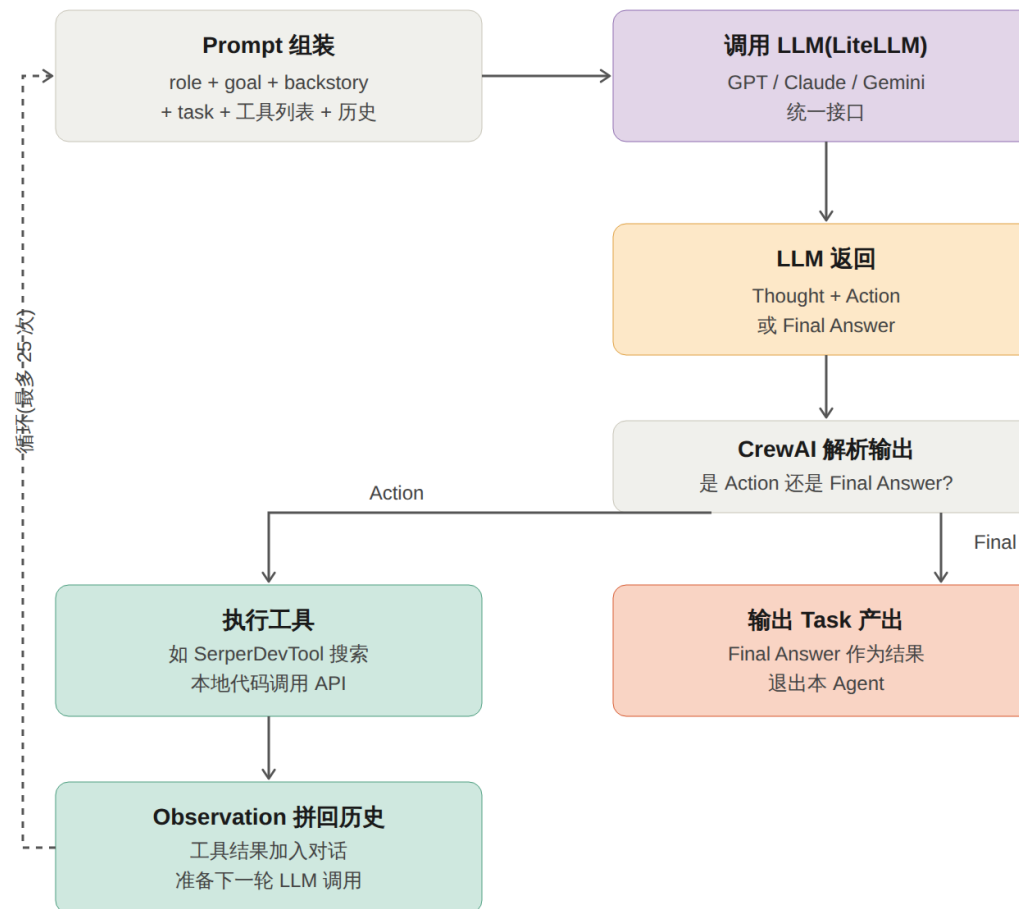


魔泊云
MoPaaS

一个 Agent 不是只调用一次 LLM,而是进入循环:



直到 **Final Answer** 或达 *max_iter*(默认 25)



两个 Agent 是怎么协作的

context 字段是真正的“胶水”



智泊AI
ZHIPU AI



魔泊云
MoPaaS

4

次 LLM 调用

2

次工具调用

1

次 context 传递

关键观察

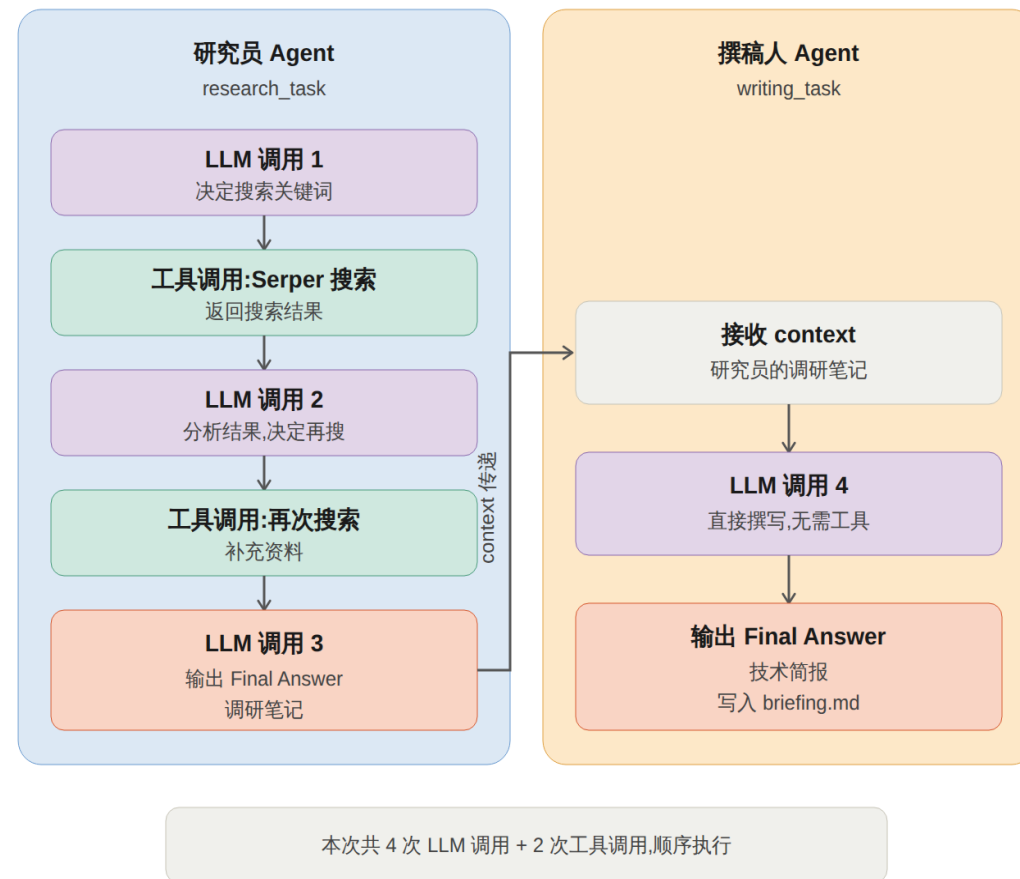
研究员有工具

→ 多次循环,每次决定再搜不搜

撰稿人无工具

→ 一次 LLM 调用就出结果

本质:不是“对话”,是“注入”



LLM 调用是怎么实现的

LiteLLM 作为统一适配层



三种配置方式(优先级从高到低)

Agent 级别	<code>Agent(..., llm="anthropic/claude-sonnet-4-5")</code>	最灵活
Crew 级别	<code>Crew(..., manager_llm="openai/gpt-4o")</code>	层级模式专用
环境变量	<code>export OPENAI_API_KEY=sk-...</code>	默认 fallback

PART 04.4

Token 经济学

Prompt 拼装 · 成本结构 · 优化策略



一次 LLM 调用的 Prompt 长什么样

两大块,九个区



智泊AI
ZHIPO AI



魔泊云
MoPaaS

System Prompt

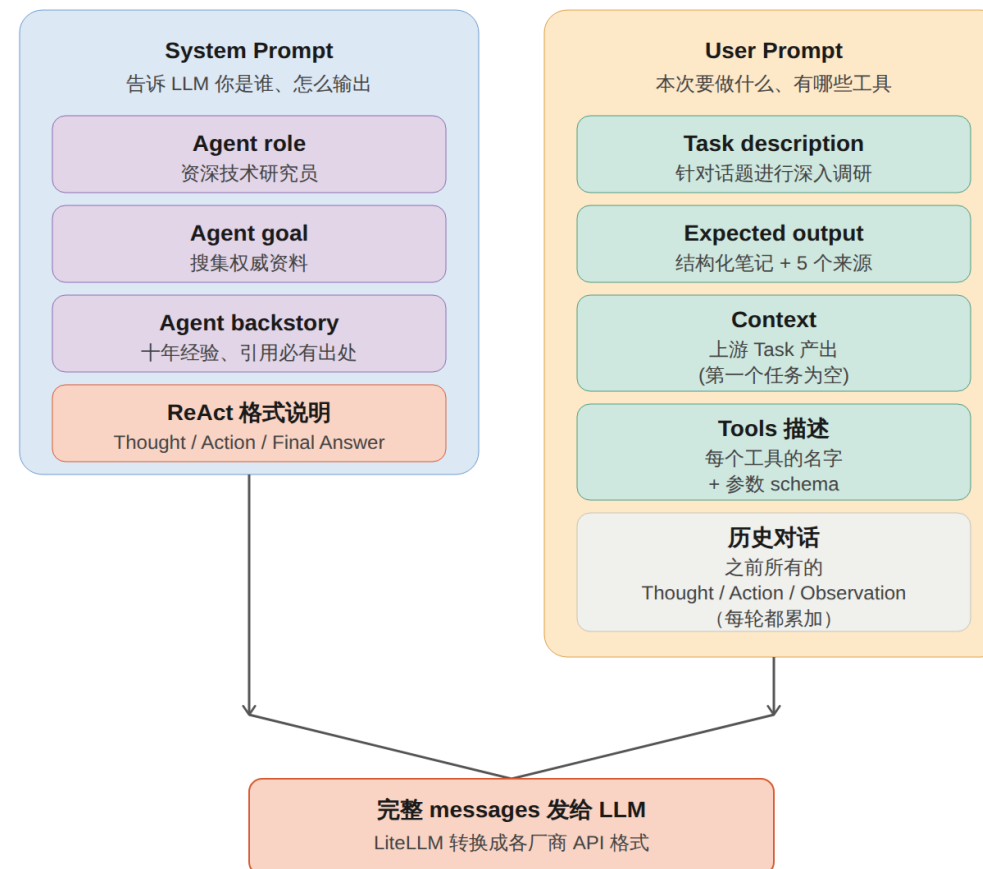
告诉 LLM “你是谁、怎么输出”

- Agent role + goal + backstory
- ReAct 格式说明

User Prompt

本次要做什么、有哪些工具

- Task description + expected_output
- Context(上游 Task 产出)
- Tools 描述
- **历史对话(每轮都增长!)**



关键观察:历史块每轮都增长,这是 token 消耗膨胀的源头

为什么 Prompt 会越来越大

LLM 是无状态的,历史必须重发



智泊AI
ZHIPO AI



魔泊云
MoPaaS

三轮 LLM 调用的 token 消耗

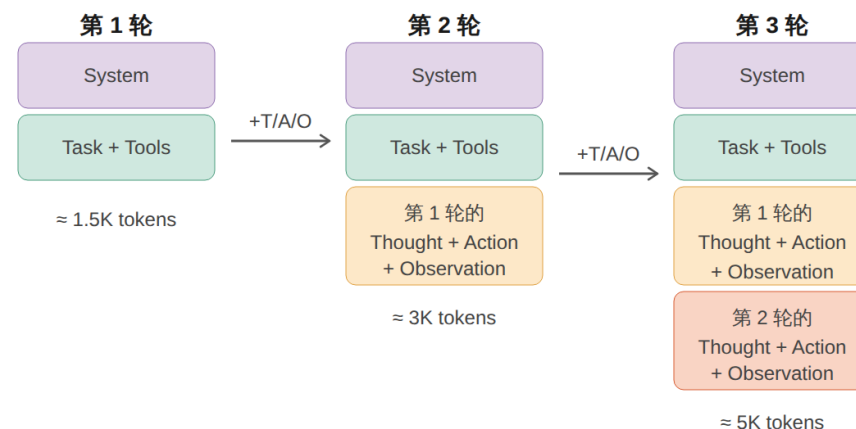
第 1 轮	1.5K	System + Task + Tools
第 2 轮	3.0K	+ 第 1 轮的 T/A/O
第 3 轮	5.0K	+ 第 1、2 轮的 T/A/O

三轮合计 $\approx 9.5K$ tokens(仅输入)

每轮都是独立 API 调用,各自计费

研究员 Agent 的三轮 LLM 调用

同色块表示同一份数据被反复发送



紫色 + 青色块每轮都重发,橙色 + 珊瑚色是新加入的历史

LLM 调用次数越多,每次的 prompt 就越长 —— 这是叠加效应

总 token 不是 $1.5K + 3K + 5K$

而是 $1.5 + 3 + 5 = 9.5K$ (输入)+ 各轮输出

因为每轮都是独立 API 调用,各自计费

三种模式的成本对比

多 Agent 不是 4 倍,是 20 倍



智泊AI
ZHIPU AI



魔泊云
MoPaaS

2K

直接问 LLM

1x

15K

单 Agent ReAct

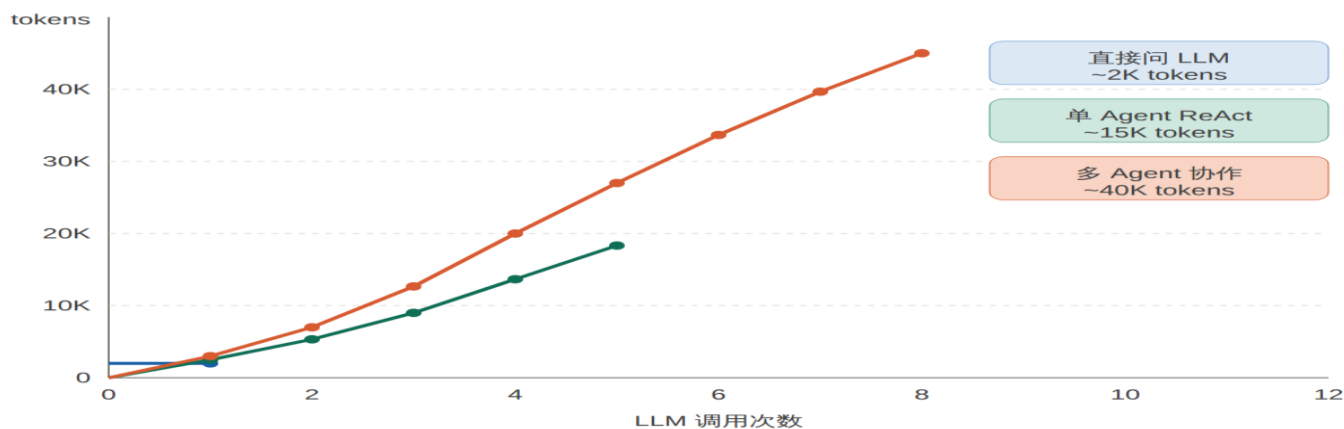
7.5x

40K

多 Agent 协作

20x

三种模式的累积 token 消耗对比(示意)



关键洞察

调用次数 × 历史累积
= 双重叠加效应

再 + 跨 Agent 的
context 传递

6 条省 Token 实战建议

理解机制后,优化就清晰了



1 Backstory 精简

几句话足矣 —— 每轮都重发

2 限定 max_iter

默认 25 太多,设 5-8 通常够用

3 只给需要的工具

工具描述也每轮重发

4 开启 prompt caching

Claude 自动缓存,省 50-90%

5 优先 sequential

hierarchical 的 Manager 也烧 token

6 简单子任务用便宜模型

Haiku 写作 + GPT-4o 调研